BUILDING R.O.B. - VOLUME 8

ENGINEERING CEREBRO

Three-camera perception, private AI, face identity, Messages, and recorded evidence

AN IMPLEMENTATION-LEVEL GUIDE FOR SWIFT AND OBJECTIVE-C DEVELOPERS

Rodolfo Aramayo / Orbitus Robotics - First Maker Faire Edition

Build a robot mind from bounded, inspectable parts

Cerebro becomes reliable when every camera frame, serial byte, model turn, and actuator request crosses an explicit contract.

This volume reads the current Cerebro repository as a living mixed-language robot system and restores its pre-v5 lineage from the preserved 2017-2025 workshop archives. It dissects ROBSerialBox and its POSIX/IOKit device paths; separate face, belly, and Insta360 camera roles; Vision and SceneKit rendering; MLX and Gemini inference; consent-based encrypted face identity with selectable AdaFace encoders; the fail-closed Messages bridge and optional encrypted memory; explicit synchronized recording; SceneSnapshot; bounded autonomy; safe shows; and the three-feed Vision Pro contract taught in Volume 7.

Paths are relative to the public `https://github.com/RudyAramayo/Cerebro` repository, with the spatial client in `https://github.com/RudyAramayo/ROBControllerVision`. Every chapter names the exact implementation or test being analyzed. Historical artifacts are not automatically recommended, model output is never motor authority, and physical behavior must be verified under a controlled commissioning procedure.

First evidence-based draft: 2 August 2026. This historical engineering edition distinguishes documented facts, builder recollection, experiments, plans, and facts not established by the available evidence.

SOFTWARE CONFIDENCE IS NOT PHYSICAL CERTIFICATION

Cerebro controls high-energy mobile and articulated hardware. A successful build, camera detection, model response, serial banner, green UI state, or passing fixture does not certify wiring, brakes, limits, calibration, collision avoidance, force, stability, or emergency stopping. Test unpowered first, isolate actuators, use conservative limits, maintain a physical emergency stop, and keep one responsible human in command.

The Building R.O.B. library

VOLUME 1	*Meet ROB: A Robot Is a Team of Systems* - ages 8-12
VOLUME 2	*Circuits and Signals with ROB* - ages 10-14
VOLUME 3	*Motion Workshop: Treads, Gears, and Fabrication* - ages 10-15
VOLUME 4	*Mission Control: Arduino, Mac, Networks, and Vision* - ages 12-16
VOLUME 5	*AI, Robotics, and Codex: Directing, Verifying, and Owning AI-Built Systems* - advanced makers
VOLUME 6	*Dual-Arm Robotics: AMBER B1, URDF, CAN, and Ubuntu* - advanced makers
VOLUME 7	*Engineering ROBControllerVision: Swift, Spatial Input, Video, and Speech* - Swift developers
VOLUME 8	*Engineering Cerebro: Perception, Models, H.264, and Robot Coordination* - software engineers
MANUAL	*Building R.O.B.: The Complete Maker's Field Manual* - advanced builders

HOW TO USE THIS BOOK

Volume 8 is the robot-side companion to Volume 7. Read Volume 7 for ROBControllerVision's Swift 6 state model, explicit controller ownership, dual-arm input, three H.264 feeds, immersive 360 presentation, and speech. Read this volume for Cerebro's matching server, camera, identity, private-conversation, recording, model, hardware, and safety architecture.

HOW TO FIND THE FILES

Open the public repositories on GitHub and follow each printed repository-relative path: `https://github.com/RudyAramayo/ROBBooks`, `https://github.com/RudyAramayo/ROBArduino`, `https://github.com/RudyAramayo/Cerebro`, `https://github.com/RudyAramayo/ROBController`, `https://github.com/RudyAramayo/ROBControllerVision`, `https://github.com/RudyAramayo/Amber-HomeFolder`, and `https://github.com/RudyAramayo/ROBTrainingGames`. The website is published separately at `https://github.com/OrbitusRoboticsWebSite/ORobotics`. See `https://github.com/RudyAramayo/ROBBooks/blob/main/ROB-Books/OPEN-SOURCE-CODE-MAP.md` for clickable repository and file links, license cautions, and renamed-file search instructions. A named archival item without a verified public repository is labeled as evidence, not given an invented URL.

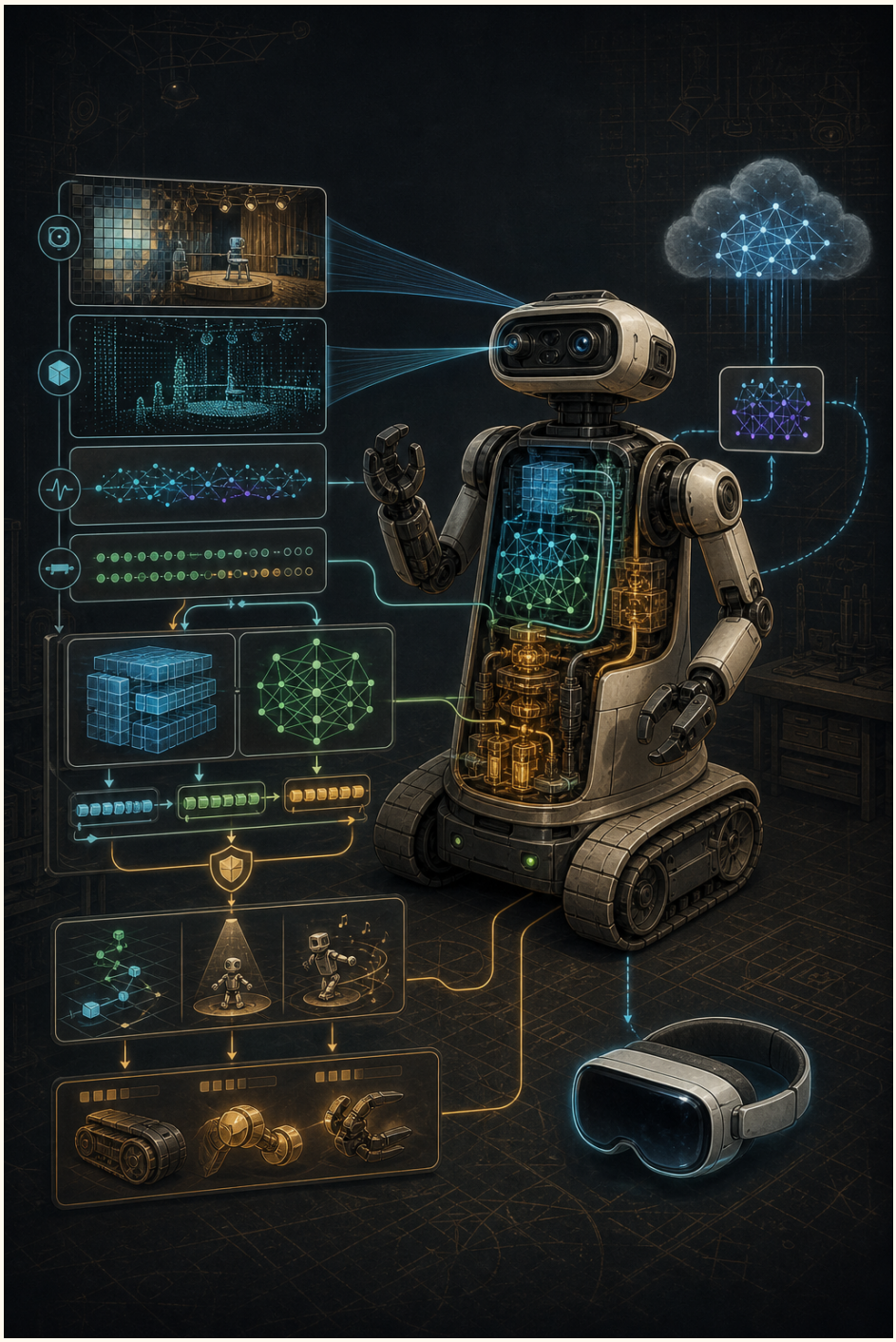

Figure 1: *Conceptual illustration: Cerebro admits camera, depth, serial, local-model, cloud-model, show, and Vision Pro data through separate bounded paths. It is a teaching diagram, not a wiring plan or literal screenshot.*

Contents

CHAPTER 1

Read Cerebro as a living robot system

Cerebro is the macOS application at the center of ROB. It is not one algorithm. It is a boundary between three camera roles, serial devices, robot controllers, speech, local models, cloud models, private Messages, consent-based face identity, explicit dataset recording, two AMBER arms, a stage-show engine, and a human operator. The public source is `https://github.com/RudyAramayo/Cerebro`. Its Git root is the 2025 v5 migration, not Cerebro's beginning; the recovered 2017-2025 lineage is documented below. Clone the public source, then use this repository root:

```
Cerebro/
```

Keep Xcode and the repository open while reading. Paths beginning `Cerebro/` are relative to that repository. Tests are under `Tests/`; operational notes are under `docs/`. The companion spatial client is in the sibling `ROBControllerVision/` repository and is taught in Volume 7.

> **SOURCE TRAIL — ANALYZING NOW:** `README.md`, `Cerebro.xcodeproj/project.pbxproj`, `Cerebro/AppDelegate.m`, and `Cerebro/ROBMainViewController.mm` establish the application boundary and composition graph.

This book uses **observed** for behavior supported by the inspected source, **historical artifact** for retained code that is not the preferred current path, and **proposed** for an improvement. Never infer that a compiled control button proves a physical mechanism is safe or calibrated.

CHAPTER 2

Recover Cerebro's pre-v5 lineage

> **SOURCE TRAIL — HISTORICAL ARCHIVE:** `CEREBRO_ARCHIVE_HISTORY.md`; the preserved `Cerebro v1` through `Cerebro v5` folders; v2 root `0141a646`; v3 master `00fbf6b`; v4 master `1a3c779`; and the independent v5 root `4c4f1d4`.

The current public repository starts on 5 August 2025, but Cerebro's surviving history reaches back to ROB-specific source dated 2017-2018 and a Git root from 1 January 2018. The v5 root commit describes itself as an initial commit with v5 changes in a fresh repository. Treat it as an import boundary.

- **v1:** pre-Git ROB additions dated 2017-2018 reveal the Mac serial command station and ROB's seven-field motor frame.
- **v2:** eleven commits in 2018 add speech, three serial roles, UI composition, multipeer, SceneKit, depth/user tracking, and Leap Motion.
- **v3:** the 2018 root and fifty commits across archived branches through 2022 preserve authority experiments, servos, multilingual speech, process-based sensors, human tracking, RTSP, T265, and AutoNet.
- **v4:** the 2018 root and fifty-four commits across archived branches through 2025 preserve Apple-silicon migration, Google LLM responses, process singleton work, continuous speech, and Foundation Models.
- **v5:** an independent 2025 Git root migrates the existing application into the history developed today.

v1: the serial nervous system

The oldest folder retains a Cocoa project derived from Gabe Ghearing's 2009 "Arduino Serial Example." That attribution belongs to the starting example; it is not Cerebro's birth date. ROB-specific headers credit Rob Makina on 18 September 2017 and 1 January 2018, and the project product is named Cerebro.

The adapted program opens a hard-coded USB modem path at 250000 baud. Its buttons issue forward, reverse, turns, flipper motion, and signed linear- actuator motion. `BotCommands` records the same seven signed fields later formalized as ROB's historical 42-byte text frame. A keyboard view and multipeer manager show the command station beginning to accept more than one kind of operator input.

This makes serial framing, not AI, the first surviving Cerebro concern. The application began by translating human intent into the exact bytes expected by a physical machine.

v2: voice, multiple bodies, and experimental senses

The first surviving Git repository begins with `0141a646`, committed by Rob Makina on 1 January 2018. On 2 January, the history completes a SpeechBox, adds working Base, Head, and Torso serial paths, binds controls and windows, and adds multipeer and SceneKit controllers.

Its main controller composes serial, speech, keyboard, SceneKit, multipeer, NiTE, Leap Motion, chat, and a class named `ROBConsciousness`. Names indicate the intended architecture; they do not prove that the early consciousness bridge or 3D view was a complete intelligence or calibrated digital twin.

January commits record OpenNI, PCL, point-cloud, and NiTE user-tracking work. One celebrates data appearing in Cerebro; another calls the integration a failure. May commits add libfreenect/NiTE work and Leap Motion. Pre-

serving both outcomes matters because later Kinect files are not mysterious debris: they are the residue of a documented effort to give the command station spatial and human input.

v3: arbitration becomes a requirement

The v3 archive retains the 2018 root and separate NiTE2, libfreenect, ROB2, and T265 experiment branches. In May 2019, commit `892cd08` requires a master controller identity so devices do not issue conflicting input, and it requires permission for autonomous mode. The modern ownership and authorization layers are a rigorous descendant of that earlier workshop problem.

The surrounding history adds Maestro servo control, mood and voice volume, multilingual speech, wireless joining, head tracking, ReSpeaker and RealSense task processes, VTK/NiTE work, visual recognition, human tracking coupled to torso camera movement, RTSP, T265 tracking, headless-performance changes, and speech/chat repair. The master branch reaches April 2022, when Cerebro adopts AutoNet, binds the iPad consciousness controller, and separates RPLidar into another repository.

v4 and v5: migration is not invention

The v4 folder shares the 2018 root but follows a branch that diverged from the later v3 master after their 9 November 2019 T265/perception checkpoint. Its master records an M1 checkpoint, safer text output, USB network work, Google LLM responses, singleton-process checking, continuous speech, and Apple Foundation Models experiments. A 2 July 2025 commit explicitly says the builder is creating the next v5 release.

The v5 root, `4c4f1d4`, has no parent. Its imported tree already contains storyboards, serial and speech systems, camera management, AutoNet, lidar, Kinect/RealSense artifacts, Leap Motion, RTSP, task launchers, and Core ML assets. The source did not appear all at once; only its new Git container did.

Preserve evidence while modernizing

Do not flatten the five archives into one invented linear history. The v3 and v4 master branches diverge, v1 has no Git metadata, and some working folders contain uncommitted or iCloud-placeholder state. Use committed objects where available, preserve original `.git` directories and third-party notices, and record hashes before moving files.

The old author labels—Rob Makina, Rodolfo Aramayo, Orbitus, and one placeholder email—also should not be used to invent contributors. They are historical Git coordinates. The source archive establishes what the program contained and when the commits occurred; a first-person oral history remains the right place to explain names, motivations, and the lived relationship between software and the physical ROB.

CHAPTER 3

Map the mixed-language architecture

Cerebro grew across generations of Apple and robotics technology. Its composition therefore crosses Objective-C, Objective-C++, C++, Swift, Python, shell helpers, JSON, Core ML models, and SceneKit assets.

- **Application shell:** `AppDelegate.m` and `ROBMainWindowController.m` own lifecycle, windows, service startup, and dependency checks.
- **Operator coordination:** `ROBMainViewController.mm` joins camera, speech, transport, shows, and robot state.
- **Physical I/O:** `ROBSerialBox.h/.m` reaches Base, Maestro, Tic, AMBER, and historical serial roles.
- **Camera and perception:** `CameraManager.swift`, `CameraViewController.swift`, and the `ROBInsta360` services separate face, belly, and panoramic provider lifecycles from Vision/SceneKit presentation.
- **Local and cloud intelligence:** `ROBMLXRuntime.swift`, `ROBAI.swift`, and `GeminiRoboticsProtocol.swift` keep provider concerns separate.
- **Private conversation:** the `ROBMessages` types isolate one-to-one text/image turns, current-information tools, optional encrypted memory, and local administrator commands from room conversation and motor tools.
- **Local identity:** the `ROBFace` types own consented enrollment, encrypted model-tagged profiles, embedding inference, and untrusted recognition context.
- **Training capture:** `ROBRecordingCoordinator.swift` owns explicit synchronized datasets and refuses autonomous self-labeling.
- **Typed world state:** `ROBSceneSnapshot.swift` describes people, objects, free space, poses, and confidence.
- **Performance:** the `ROBStageShow` files and `ROBSaberChoreography.swift` validate cues and gesture names.
- **Spatial remote:** the sibling `ROBControllerVision/` repository owns Vision Pro input, video decoding, and operator UI.

Objective-C owns much of the hardware-facing application because it grew from Cocoa-era controller code. Swift supplies newer protocol, concurrency, machine-learning, and media boundaries. The bridging header is therefore architectural infrastructure, not a temporary embarrassment. Keep exposed interfaces narrow, nullability explicit, and callbacks asynchronous.

CHAPTER 4

Reproduce the build before changing behavior

SOURCE TRAIL — ANALYZING NOW: `Package.resolved` inside the Xcode workspace, plus `Cerebro/PythonRequirements.txt` and `README.md`.

The project pins MLX-family packages. The inspected documentation records `mlx-swift` 0.31.3, `mlx-swift-lm` 3.31.3, `swift-tokenizers-mlx` 0.3.0, and `swift-hf-api-mlx` 0.2.0. Pinning matters: model-loading APIs, tokenizers, Metal kernels, and model configurations evolve together.

```
xcodebuild -project Cerebro.xcodeproj \
  -scheme Cerebro -configuration Debug \
  -destination 'platform=macOS' \
  -derivedDataPath /tmp/CerebroDerivedData \
  CODE_SIGNING_ALLOWED=NO build
```

Install the Xcode Metal component if MLX compilation requests it. Configure DepthAI through Cerebro's Python Settings rather than inserting a personal interpreter path. Download local models before an offline show and test cold load, warm load, sustained memory, thermal behavior, camera reconnect, and stop behavior on the production Mac.

CHAPTER 5

Enter through AppDelegate, then find ownership

`AppDelegate` is where process-lifetime services belong: the singleton/supervisor handshake, secure control/video listeners, headless camera helper supervision, Messages and face services, optional dependency discovery, wake recovery, and coordinated shutdown. `ROBMainViewController` connects those services to windows and UI state. `ROBSerialBox` owns hardware channels. `CameraViewController` owns presentation and perception work, while `CameraManager` owns camera-provider mechanics.

The production LaunchAgent uses a crash-limited supervisor, while Xcode development performs an intentional production/debug handoff. The process acquires its singleton lock before AppKit construction so two copies cannot compete for cameras, serial ports, Messages high-water state, or network listeners. Wake is a new health boundary: refresh camera, model, connection, and helper state rather than assuming resources survived sleep.

The essential ownership questions are:

1. Who starts this resource?
2. Who stops it on window closure, disconnect, camera replacement, and app termination?
3. Which queue or actor mutates it?
4. How are late callbacks from a retired generation rejected?
5. What bounded state survives when the consumer is slower than the producer?

A robot app should not rely on `deinit` as its only shutdown protocol. Cameras, file descriptors, subprocess pipes, Network.framework connections, timers, model tasks, and SceneKit nodes all need explicit lifecycle edges.

CHAPTER 6

Dissect ROBSerialBox: the hardware boundary

> **SOURCE TRAIL — ANALYZING NOW:** `Cerebro/ROBSerialBox.h` and `Cerebro/ROBSerialBox.m`, especially `initialize_connection`, `usbSerialPortPaths`, `connectToDetectedBase`, `probeBaseFirmwareAtPath`, `consumeBaseSerialBytes`, `renderController`, and `stopBaseMotionAndDropHeartbeat`.

`ROBSerialBox` is both valuable and overloaded. Its header exposes serial selection, tread and flipper controls, neck and torso commands, Pololu Maestro and Tic operations, two AMBER-arm command families, controller snapshots, and legacy Head/Torso surfaces. Treat it as an archaeological map of physical integration.

At initialization it sets every descriptor to `-1`, initializes demanded and actual speeds, creates a bounded Base receive buffer, begins Base discovery asynchronously, identifies the Maestro, starts a 100 ms controller timer, and later starts AMBER log-tail helpers. The retired Head and Torso Arduino UI remains visible, but automatic opening is commented out because only Base is presently installed.

File descriptors are state machines

A descriptor is not merely an integer. It moves through closed, opening, configured, readable/writable, failed, and closing states. `-1` is the closed sentinel. Every read and write must use a stable ownership rule; otherwise a reconnect can close a descriptor while an old thread is writing, producing the historical `Bad file descriptor` flood.

The current low-level serial configuration uses POSIX `open`, `termios`, `select`, `read`, and `write`. Cocoa objects may format diagnostics, but blocking I/O must not run on the main thread. A modern refactor should wrap each channel in a serial executor or actor-like Objective-C object and expose typed commands rather than public descriptor-dependent methods.

```
if (serialFileDescriptor != -1) {
    write(serialFileDescriptor, bytes, length);
}
```

That check alone does not make concurrent close/write atomic. The durable pattern is one queue owning open, read, write, and close, plus an incrementing connection generation attached to callbacks.

CHAPTER 7

Discover the Base Arduino by firmware response

Cerebro enumerates IOKit serial services, accepts only `/dev/cu.usbmodem*` and `/dev/cu.usbserial*`, and sorts candidates. It opens each candidate at 250,000 baud, pulses DTR to reset an Arduino, sends no probe command bytes, and waits up to 15 seconds for the existing startup line:

```
BEGIN BASE STARTUP SEQUENCE
```

Only the matching descriptor becomes `serialFileDescriptor_base`. This is better than remembering a `/dev/cu.*` name because hub topology can rename ports. It also avoids sending an accidental motion-like byte to an unidentified device. The startup string is role discovery—not authentication, wiring validation, or proof that motion is safe.

The probe retains at most 8 KiB and trims older bytes. The running line parser caps unterminated input similarly. Those small limits matter: a noisy serial device must not grow application memory forever.

Parse lines before interpreting meaning

`consumeBaseSerialBytes` accumulates arbitrary read chunks, extracts newline-delimited records, and then calls `handleBaseSerialLine`. This respects a fundamental serial fact: one `read` is not one application message.

The parser recognizes existing warning strings such as `WARNING! FRONT` and `WARNING! BACK`. It also accepts a proposed numeric six-sensor line beginning `ROB:IR=` when available. Each field must contain only decimal digits and be at most 1000. Legacy warnings expire visually; silence means **clearance unknown**, never **path clear**.

CHAPTER 8

Separate devices by identity and protocol

Base discovery uses a firmware banner because that is what the existing sketch emits. Maestro discovery instead inspects USB vendor, product, interface name, and interface number, preferring the named command port. The Tic waist-rotation path invokes validated `ticcmd` arguments. These are three different identity strategies because the devices expose different trustworthy evidence.

Do not create a universal "pick the first USB port" helper. Build a device registry whose match result includes device role, evidence, path, generation, capabilities, and last error. A UI can then report "Base verified by startup banner" and "Maestro matched by USB identity" instead of showing a misleading green dot.

CHAPTER 9

Translate controller intent without extending stale motion

`controllerPassthrough` turns controller state into the legacy Base frame and actuator targets. `renderController` runs at 10 Hz while snapshots are expected at 5 Hz. Cerebro expires controller snapshots after three missed updates, writes one neutral/braked frame, and then stops Base USB writes so the Arduino's own heartbeat deadman can expire.

This is a two-layer timeout:

```
controller freshness expires
        ↓
Cerebro emits one neutral frame
        ↓
Cerebro stops refreshing Arduino heartbeat
        ↓
Base firmware independently de-energizes motion
```

Never solve packet loss by endlessly replaying the last nonzero command. The more powerful the actuator, the more important command age, authority owner, and stop semantics become.

CHAPTER 10

Control Maestro neck servos and Tic torso rotation

> **SOURCE TRAIL — ANALYZING NOW:** `ROBSerialBox.m` methods `applyVisionNeckPan:tilt:`, `applyVisionTorsoActive:rotation:`, `waistRotationSliderAction:`, `exitSafeStartWaistRotationToggle:`, and `energizeToggle:`.

Vision Pro head orientation arrives as normalized intent. Cerebro clamps it, maps it into Maestro target units, avoids redundant writes, and sends compact protocol bytes. Torso follow maps bounded orientation into Tic position units. The Tic must exit safe start and be energized before accepting motion; the UI mirrors those states and its full rotation range.

Commanded position is not measured position. On boot, an actuator controller can believe a coordinate origin that does not match the physical pose. The book's rule is therefore:

```
commanded target ≠ encoder feedback ≠ visually estimated pose
```

Store each with source, timestamp, confidence, and calibration generation. Never overwrite one with another just because they share units.

CHAPTER 11

Bridge Objective-C and Swift deliberately

SOURCE TRAIL – ANALYZING NOW: `Cerebro/Cerebro-Bridging-Header.h`, `ROBMainViewController.mm`, `ROBAI.swift`, and the `@objc` entry points in newer Swift services.

Objective-C callers need stable selector-shaped APIs and Foundation-compatible types. Swift implementation types can keep actors, `Sendable`, async functions, and enums internally, then expose a façade that translates callbacks and notifications.

Use these boundaries:

- cross with `NSString`, `NSData`, `NSNumber`, `NSArray`, `NSDictionary`, and explicitly Objective-C-compatible classes;
- turn Objective-C delegate events into immutable Swift values immediately;
- dispatch UI mutation to the main actor/main queue;
- never expose an MLX tensor, `CVPixelBuffer` lifetime assumption, or actor-isolated mutable object through the bridge;
- make ownership weak where the caller and service retain one another;
- give errors a bounded, user-safe category and keep secrets out of descriptions.

CHAPTER 12

Capture RGB with AVFoundation

SOURCE TRAIL — ANALYZING NOW: `Cerebro/CameraManager.swift` types `CameraSource`, `CameraSourceState`, `CameraFrameSet`, and its AVFoundation capture methods.

The AVFoundation path discovers a camera, chooses a format, configures `AVCaptureSession`, and receives `CMSampleBuffer` frames. Configuration should occur inside `beginConfiguration/commitConfiguration`; capture callbacks belong on a dedicated queue; UI work belongs on the main actor.

`CMSampleBuffer` preserves image buffer and timing. Keep it intact as long as possible. Converting every frame into `NSImage`, then `CIImage`, then a new pixel buffer creates copies and allocation churn. Consumers should receive one frame set and select the cheapest representation they actually need.

Camera lifecycle is demand-driven. Local preview, Gemini sampling, MLX vision, and Vision Pro streaming may each create demand. The session should run when at least one demand exists, not restart every time a checkbox changes. A run generation prevents callbacks queued by an old session from contaminating a replacement session.

CHAPTER 13

Acquire aligned RGB-D from Luxonis

SOURCE TRAIL — ANALYZING NOW: `docs/depth-camera.md`, `Cerebro/Webcam_color.py`, `CameraManager.swift`, `ROBPythonRuntime.m`, and `Tests/DepthCameraIPCFixtureTests.py`.

The primary OAK path uses DepthAI 3.8.0 in a supervised Python helper. The helper alone owns one device, builds Camera + Depth + Sync nodes, requests the selected main-camera profile (1280 by 720 by default, with 640 by 400 available), aligns depth to RGB, and sends paired frames through a private Unix-domain socket. The face and belly roles have distinct MXIDs, socket names, lifecycle state, and on-device graphs; they must never compete for one socket or physical OAK.

The versioned `CDP1` packet contains bounded metadata, RGB888 bytes, and little-endian unsigned 16-bit depth in millimeters. Zero depth is invalid. Cerebro validates magic, version, dimensions, lengths, pixel formats, and maximum allocation before creating CoreVideo objects.

The out-of-process boundary is a reliability feature. USB disconnects, SDK exceptions, malformed packets, and native-library faults disable/restart the camera provider without taking robot control down. Retries use bounded backoff. AVFoundation remains an RGB-only fallback, while legacy OAK UVC fallback is explicit opt-in so two providers never fight for the same device.

The builder identifies two present Luxonis roles: an OAK-D Pro Wide in the body/belly and an autofocus OAK-D Pro at the head/face. Do not infer the exact sensor option from the enclosure. At discovery, save the intended role, MXID, `productName`, `boardName`, board revision, camera sensor names, USB speed, calibration hash, focus capability, and firmware/API version. The RVC2 OAK-D Pro W family uses global-shutter monochrome stereo sensors and offers a wide color-camera option; its active-stereo projector and flood illumination add power and thermal load. A camera that enumerates successfully can still be on a USB 2 link, underpowered, thermally constrained, out of calibration, or assigned to the wrong role.

The Insta360 Pro II is a third, independent role. `ROBInsta360CameraService` establishes the camera control session, maintains heartbeat, supervises RTMP/ffmpeg preview, and reports bounded retry state. `ROBInsta360PerceptionService` can analyze the panorama as six sectors without requiring the diagnostics window to remain open. Decode remains demand-driven: headless perception, recording, flat preview, and immersive streaming declare their needs so closing an unused window can actually release work.

Separate host inference from OAK inference

ROB has three distinct model execution paths:

1. **DepthAI device graph:** camera, stereo depth, synchronization, and an optional compiled Myriad X `.blob` execute in the OAK pipeline. Only bounded results and RGB-D frames cross the USB process boundary.
2. **Apple Vision/Core ML:** `ROBDynamicDetectorRegistry` runs built-in Vision requests or registered Core ML models on the Mac. It compiles an `.mlmodel` to `.mlmodelc` when needed, creates `MLModel`, wraps it in `VNCoreMLModel`, and executes `VNCoreMLRequest` at a configured admission rate.
3. **MLX language/vision models:** actor-owned local LLM, VLM, and embedding containers run on Apple silicon for delayed reasoning and stage context, never the motor loop.

The same word *model* appears in all three paths, but their files, devices, input tensors, output schemas, latency, memory, and failure scopes differ. Record those facts in every model manifest.

Understand alignment

Depth alignment means pixel `(x,y)` in the depth plane refers to the same camera ray as pixel `(x,y)` in RGB after calibration and resampling. It does not mean every depth is valid. Reflective, transparent, too-near, too-far, or texture-poor surfaces can return zero or noisy values.

`CameraDepthFrame.distanceMillimeters(x:y:)` validates coordinates and byte offsets. Downstream code rejects implausible distances. A person's estimated range is the median of a small interior grid, not a single fragile pixel.

CHAPTER 14

Build a bounded perception pipeline

SOURCE TRAIL — ANALYZING NOW: `CameraViewController.swift`, `HumanBodyPose3DDetector.swift`, `HumanBodySkeletonRenderer.swift`, and `ROBSceneSnapshot.swift`.

Camera rate is not inference rate. A 20–30 fps camera may feed a body request only when the previous request has finished. The UI may display the newest RGB frame while depth visualization, pose inference, MLX VLM sampling, Gemini JPEG encoding, and remote H.264 encoding operate at different bounded rates.

The reliable pattern is a latest-value mailbox:

```
producer offers frame N
worker busy? replace pending frame, do not append
worker finishes
consume newest pending frame, if any
```

This bounds latency and memory. A robot benefits more from the newest frame than from processing a five-second-old queue perfectly.

Vision observations are normalized and carry confidence. SceneKit nodes should be reused or replaced as a group, not appended forever. Heavy geometry creation stays off the main thread where APIs allow; final scene mutation is serialized. Wrap temporary per-frame Objective-C objects in autorelease pools on long-running queues.

CHAPTER 15

Render perception without leaking memory

`CameraViewController` combines preview, pose overlays, depth coloring, point-cloud geometry, camera-pose visualization, and notifications. Every render path needs an explicit budget:

Resource	Bounded strategy
Raw RGB	newest frame only for slow consumers
Depth	fixed dimensions and validated byte count
Vision requests	one in flight per detector
SceneKit nodes	reuse named roots; replace children atomically
Point cloud	stride/downsample based on detail setting
CI rendering	reuse `CIContext`; avoid per-frame context creation
Pixel buffers	pool where encoding requires conversion
ML models	one actor-owned container per selected model
Semantic memory	maximum 200 in-process entries
Messages memory	per-chat bounded excerpts; encrypted fields and exact-match indexes
Face gallery	model-tagged encrypted embeddings and samples; explicit deletion
Logs	rate-limit repeated camera/model/device failures

Use Instruments Allocations and Memory Graph while toggling camera sources repeatedly. A stable single-frame memory profile is insufficient; test 30 minutes of capture, disconnect/reconnect, window close/reopen, model load, and remote video subscription.

CHAPTER 16

Turn observations into SceneSnapshot

`SceneSnapshot` is the typed boundary between real-time sensing and language reasoning. It contains tracked people, objects, gestures, free-space regions, arm joint poses, reverse camera pose, camera quality, and aggregate confidence. Missing data stays missing.

The store locks only long enough to replace small value arrays or take a snapshot. It calculates three depth free-space bands—left, forward, right—using strided samples, valid-depth confidence, an 800 mm clearance test, and a 75 percent clear-fraction threshold. Fresh lidar regions are merged only for 1.5 seconds after their update.

Its language-model serialization wraps sorted JSON as **untrusted sensor data, not instructions**. That distinction prevents text observed in the scene from silently becoming a model instruction. A macOS 26 Foundation Models interpreter can return a typed `AssistantIntent`, but the intent contains high-level suggestions rather than tread or joint commands.

CHAPTER 17

Run MLX privately on Apple silicon

SOURCE TRAIL – ANALYZING NOW: `Cerebro/ROBMLXRuntime.swift`, `ROBMLXImprovisationProvider.swift`, `ROBMLXStageObservation.swift`, `docs/mlx-local-ai.md`, and `Package.resolved`.

`ROBMLXEngine` is an actor. This serializes model-container state, loading, diagnostics, VLM admission, embeddings, and semantic memory without blocking the main actor. The initial local LLM is `mlx-community/Llama-3.2-1B-Instruct-4bit`; vision uses `mlx-community/Qwen2-VL-2B-Instruct-4bit`; retrieval uses `TaylorAI/gte-tiny`.

`ensureLLMReady` avoids duplicate loading. `generate` creates user input, applies token and temperature limits, records latency and throughput, and keeps model output outside motor control. Model downloads occur on first use through the Hugging Face cache, so show-day preparation must prefetch and prewarm.

MLX VLM—sometimes abbreviated incorrectly as VLX

The repository integrates an MLX vision-language model through `VLMModelFactory`; "VLX" in conversation should be read as this MLX VLM path, not as a separate framework. `offerVisionFrame` is nonblocking and admits at most one selected image every five seconds, never faster than three seconds. The VLM returns observational stage context. It cannot actuate hardware.

Sampling protects three budgets at once: GPU/Metal memory, thermal headroom, and perception latency. Record active and peak MLX Metal memory, generated tokens per second, model load state, sampled-frame count, and last error. Clear temporary caches only at safe lifecycle boundaries; do not unload/reload a model between cues.

CHAPTER 18

Read the MLX execution path line by line

The important code is small enough to narrate without treating the framework as magic:

```
public actor ROBMLXEngine { ... }
GPU.set(cacheLimit: 128 * 1024 * 1024)
GPU.set(memoryLimit: 6 * 1024 * 1024 * 1024)
let container = try await loadLLM(modelID: modelID)
let input = try await container.prepare(input: UserInput(prompt: prompt))
let stream = try await container.generate(input: input, parameters: ...)
for await event in stream { ... }
```

1. `actor` makes the engine the single owner of mutable model state. It does not make every referenced framework type automatically thread-safe.
2. Cache and memory limits bound MLX's use of unified memory so vision, UI, encoding, and control retain headroom. They are caps, not reservations or proof against system memory pressure.
3. `loadLLM` returns the already loaded container when the model ID matches, otherwise it records loading/download diagnostics and builds a container through `LLMModelFactory`.
4. `prepare` tokenizes the prompt and constructs the model input. Prompt text is data; length, provenance, and privacy must be controlled before this line.
5. `GenerateParameters` bounds output tokens and temperature. Low temperature reduces randomness but does not create truth.
6. `generate` returns an asynchronous event stream. `.chunk` appends text, `.info` records throughput, and `.toolCall` is rejected in the local improvisation path.
7. `Task.checkCancellation()` prevents an abandoned stage cue from consuming unlimited time, while request correlation prevents its late text from controlling a replacement cue.
8. The returned string is still untrusted. A separate strict codec must decode an allow-listed schema before a behavior layer can even consider it.

The VLM path adds an image but keeps the same boundary. `offerVisionFrame` checks a global enable, one-in-flight flag per source, and a minimum interval. It returns immediately; the actor later prepares `UserInput(prompt:images:)`. The prompt demands one minified JSON object, the codec extracts only that object, validates types and bounds, strips person identity, attaches confidence and time, and publishes delayed context. `currentStageContext` refuses old or low-confidence observations and explicitly labels accepted facts as uncertain, non-executable context.

CHAPTER 19

Teach ROB to recognize a new object without teaching it to trust itself

SOURCE TRAIL — ANALYZING NOW: `ROBDatasetManager` in `ROBSceneSnapshot.swift`, `TrainProjectModel.py`, the model-manifest loader and `YoloSpatialDetectionNetwork` branch in `Webcam_color.py`, and `ROBDynamicDetectorRegistry.swift`.

The repository already sketches a learning-and-compilation chain:

```
human chooses project and class
  -> bounded JPEG + normalized YOLO label
  -> dataset folders + classes.txt + data.yaml
  -> YOLOv8n fine-tuning on Apple MPS
  -> best.pt
  -> ONNX export, opset 12, input 640x400
  -> blobconverter, six SHAVEs
  -> yolov8_<project>_6shave.blob + JSON manifest
  -> DepthAI helper restart
  -> YoloSpatialDetectionNetwork on OAK-D Pro
```

The dataset manager first validates a project name, normalizes the class label, rejects non-finite or out-of-bounds boxes, JPEG-encodes the selected image, writes image and YOLO label atomically, and updates class metadata. A label line is `class_index center_x center_y width height`, all normalized to the unit square. This is annotation, not learning yet.

`TrainProjectModel.py` then performs these decisions:

- **`safe_project_name`:** prevents path syntax in a project identifier.
- **`require_within`:** rejects dataset paths that escape the selected root after resolution.
- **`validated_class_names`:** requires contiguous unique class IDs and exact `nc` agreement.
- **`validate_dataset`:** requires distinct train/validation image directories, labels, and matching `classes.txt`.
- **`YOLO("yolov8n.pt")`:** starts from pretrained weights; record their exact hash and license.
- **`model.train(... epochs=50, imgsz=640, device="mps")`:** fine-tunes on the Mac; fix random seeds and save the environment and hyperparameters for reproducibility.
- **Export `best.pt` to ONNX:** creates an interchange graph; validate its outputs against the PyTorch model.
- **`blobconverter.from_onnx(... shaves=6)`:** compiles for Myriad X; compilation success is not accuracy evidence.
- **Write the JSON manifest:** binds labels, parser type, input geometry, class count, and thresholds to the blob.
- **Replace the final blob and manifest:** currently makes the artifact discoverable; production needs a signed staged release and rollback.

There is an honest integration gap: the dataset manager currently writes both `train` and `val` paths to `./images/train`, while the trainer correctly rejects identical training and validation directories. This prevents the automatic chain from being a trustworthy one-button learner. Fix the data lifecycle first: capture separate train, validation, and locked test sets by collection session or scene, so nearly identical video frames cannot leak across splits.

The DepthAI helper resolves `yolov8_<project>_6shave.blob`, loads the adjacent manifest, checks that the manifest stem matches the filename, and configures a `YoloSpatialDetectionNetwork`. Camera settings can request a restart to hot-swap the model. That restart is a deployment boundary and should happen only while the affected perception feature is unavailable and motion policy no longer depends on its results.

`ROBDynamicDetectorRegistry.registerCoreMLModel` is a separate Mac-side path. If the URL is not already `.mlmodelc`, `MLModel.compileModel` compiles it; `MLModel(contentsOf:)` loads it; `VNCoreMLModel(for:)` adapts it to Vision; a lock publishes it to future requests. The current method does not yet enforce signer, manifest, input geometry, output schema, class allowlist, benchmark, duplicate identity, or rollback. Treat it as a development hook until those checks exist.

A safe future learning architecture

ROB should never train on an observation and immediately let the new model steer or move an arm. Use a promotion pipeline:

1. **Consent and purpose:** collect only authorized images for one named task; retain provenance and deletion controls.
2. **Annotation review:** a human checks boxes/classes and removes sensitive or ambiguous samples.
3. **Split by event:** keep train, validation, and locked test scenes disjoint; add hard negatives and the surfaces/lighting ROB will meet.
4. **Reproducible training:** pin code, base weights, dependencies, seed, hardware, hyperparameters, and dataset hashes.
5. **Cross-runtime comparison:** compare PyTorch, ONNX, compiled OAK blob, and/or Core ML outputs on the same golden images.
6. **Acceptance gates:** require per-class precision/recall, confusion matrix, calibration, latency, memory, thermal, and adversarial/edge-case results against declared thresholds.
7. **Signed candidate:** package model, manifest, metrics, training record, limitations, and signer; never overwrite the last known-good artifact in place.
8. **Shadow mode:** run the candidate beside the approved model without granting control authority; log disagreements with bounded, privacy-reviewed samples.
9. **Human promotion:** a named reviewer promotes one version during a disarmed maintenance state.
10. **Rollback and expiry:** health monitoring can disable the candidate immediately; the operator can restore the prior signed release; model validity expires when camera calibration, geometry, or mission changes.

The dynamic intelligence is the system that can create, test, explain, select, and revoke a model. The model itself remains one fallible component.

CHAPTER 20

Record training evidence without letting autonomy label itself

SOURCE TRAIL — ANALYZING NOW: the recording coordinator, recording window, operational note, and both recording test suites. The source map gives each exact path.

Recording is explicit, recoverable, and separate from ordinary camera preview. One synchronized session may include face/belly RGB keyframes, aligned lossless depth, stereo frames, calibration, exact `.rscan` lidar, local pose and odometry, tread/authority state, and traversability labels. Separate face, belly, and Insta360 `.mov` files preserve high-resolution footage without pretending their encoded dimensions equal the sensor's requested or observed dimensions.

The coordinator appends durable session records so a crash does not silently turn a partial dataset into a complete one. It stores requested, observed, and encoded geometry separately and associates every sample with source, timing, calibration, and authority context. Capacity checks, retention, bystander consent, deletion, and export remain release-policy work rather than properties inferred from file existence.

Autonomous commands may be recorded as events, but they never create ground-truth labels. Belly-camera traversability can become a candidate label only after manual traversal and odometry confirmation. This breaks the dangerous loop in which a planner declares its own action safe and then trains on that declaration.

CHAPTER 21

Recognize a consented face without converting identity into authority

SOURCE TRAIL — ANALYZING NOW: the face gallery, recognition service, embedding model, identity window, AdaFace installer, identity note, and face fixtures. The source map gives each exact path.

The face system is disabled by default and remains local. Enrollment requires a paired, non-revoked operator plus confirmation at ROB. The operator collects 24 varied samples rather than one flattering photograph. Vision supplies face rectangles, landmarks, and quality gates; the embedding service normalizes an aligned crop; open-set distance/margin gates and temporal consensus decide whether a name may enter scene context.

The gallery uses opaque UUID directories. Profile metadata, embeddings, and retained JPEG samples are AES-GCM encrypted with a device-only Keychain key, and deletion removes the complete profile. Recognition context expires after 15 seconds and is serialized as untrusted sensor data. Even a profile named "administrator" provides personalization only: it cannot grant controller ownership, approve a model action, reveal secrets, run a script, or move any mechanism. Prompt-based VLM person labels are likewise not credentials.

Use the selected AdaFace encoder consistently

Two IR18 Core ML backends are installed through the repository script:

- **AdaFace R18 WebFace4M** is the recommended default and broader enrollment comparison base. Its checkpoint SHA-256 begins `7a789f6696e5`; the exact full digest is recorded in `SOURCE_SNAPSHOT.md` and the local model manifest.
- **AdaFace R18 VGGFace2** is the optional comparison backend. Its checkpoint SHA-256 begins `2360a615b119`; the exact full digest is recorded in the same two places.

Both consume a square 112 by 112 BGR crop transformed by `pixel * (2/255) - 1` and return a 512-value L2-normalized embedding. The installer converts FP16 Core ML packages, validates agreement against PyTorch, compiles `.mlmodelc`, and records hashes without committing checkpoints or weights to Git. The inspected conversions reached cosine agreement above 0.99999, and both compiled runtimes produced unit-normalized 512-value outputs.

The encoder ID is part of every enrolled profile. Never compare a WebFace4M embedding with a VGGFace2 embedding. Switching models means switch back to the profile's encoder or delete and re-enroll. The current default maximum cosine distance of 0.35 and required best/runner-up margin of 0.06 are starting points, not universal biometric guarantees. Measure false accepts, false rejects, lookalikes, lighting, pose, occlusion, printed/screens replay, and time-separated sessions on ROB. Liveness/depth defenses remain future work.

CHAPTER 22

Add bounded semantic memory

`remember` embeds short text. `retrieve` embeds a query, calculates cosine similarity, sorts matches, and returns a limited result set. The current store holds at most 200 entries and is process-local.

This is retrieval, not truth. It is still process-local and distinct from two newer persistent stores: the encrypted face gallery and the optional encrypted per-sender Messages transcript store. Those stores have narrower purposes and deletion controls; neither should be merged into general semantic memory. Never store raw camera images or secrets merely because embeddings feel abstract.

CHAPTER 23

Constrain local model output

`ROBMLXImprovisationProvider` asks for exactly one JSON document with a fixed schema, allow-listed beat, allow-listed delivery, and bounded offline line. It then decodes and validates the output independently. Prompt instructions are a generation aid; the codec is the trust boundary.

```
{
  "schema": "com.orbitusrobotics.local-improvisation-plan",
  "version": 1,
  "beat": "robot_joke",
  "delivery": "deadpan",
  "offline_line": "My timing is local; your laughter is distributed."
}
```

Cancellation is correlated by request ID. A completion that arrives after cancellation is discarded. Health checks load the selected model without granting it a physical tool. The alternate llama.cpp provider uses the same domain protocol and loopback HTTP, preserving the stage coordinator and validation logic.

CHAPTER 24

Integrate Gemini Live without coupling safety

SOURCE TRAIL — ANALYZING NOW: `Cerebro/ROBAI.swift`, `GeminiRoboticsProtocol.swift`, `ROBGeminiDiagnosticsWindowController.swift`, `docs/gemini-robotics-live.md`, and `Tests/GeminiRoboticsProtocolFixtureTests.swift`.

`ROBAI` is the Objective-C-facing façade over a Swift live-session actor. Configuration comes from explicit environment settings and credentials. Connection, microphone streaming, camera streaming, and the optional robot-action tool are separate effective switches.

The audio adapter converts captured samples to the PCM format required by the live protocol and preserves event ordering. The video adapter samples frames and produces bounded JPEG data. Neither should enqueue unlimited media. Diagnostics report effective input mode, counters, transcription state, model turns, and redacted errors without logging credentials or private payloads.

The secure WebSocket setup, realtime messages, transcription fragments, tool calls, interruptions, deadlines, and turn completion are parsed into typed events. A reconnect starts a new session generation; late callbacks from the old socket cannot complete the new turn.

Use `GEMINI_EPHEMERAL_TOKEN` where an issuing service exists; a development `GEMINI_API_KEY` is supported. Never place either in source, a show JSON file, a screenshot, or a book. Camera and microphone remain off unless the operator explicitly enables them.

CHAPTER 25

Add private Messages without importing the robot-control surface

SOURCE TRAIL — ANALYZING NOW: the Messages bridge, responder, vision policy, current-information service, transcript store, operational note, and fixtures. The source map gives each exact path.

The bridge is disabled by default and reads the local Messages `chat.db` only after the operator grants Full Disk Access. It uses query-only database access and needs Automation permission only to send a reply. Exact canonical sender allowlists, a persisted high-water mark, and at-most-once processing prevent a restart from replaying old conversations. Outgoing items, groups, reactions, partial/stale messages, unexpected account mappings, and over-rate traffic fail closed; the global limit is five accepted messages per minute.

Each allowed one-to-one chat receives an isolated AI session. The profile has no room microphone, camera, motor/action, Music, arbitrary file, or device tools. It may use Google Search on the Gemini path plus fixed-publisher read-only news and Open-Meteo weather with an explicit location. It does not expose the Mac's current location or fetch caller-supplied URLs.

One JPEG, PNG, HEIC, or HEIF attachment may be accepted up to 10 MB and 24 megapixels, then normalized to a metadata-free image no larger than 2048 pixels. Gemini may receive that normalized image when explicitly configured. Otherwise Swift MLX receives pixels and Apple Foundation Models receives only bounded textual visual analysis, never pixels. A generic answer is rejected when the reply claims image grounding it did not actually receive.

Optional transcript memory defaults off. It stores encrypted fields in SQLite, exact-match HMAC indexes, and a 256-bit device-only Keychain key under owner-only permissions. Retrieval is bounded to recent/relevant entries from the same canonical sender and account and is labeled private, untrusted context. Gemini use sends selected text excerpts off the Mac, never stored image pixels. A local browser provides search, plaintext export, and clear-all controls; those operations need an explicit retention and consent policy.

Exact one-to-one messages from locally configured administrator handles may enter a deterministic command path before AI. The default `Shutdown` and `Reboot` actions ask the same sender in the same chat for exact `YES` confirmation within 90 seconds; confirmation is one-shot. Scripts use a fixed `/bin/zsh -f -s`, receive reviewed script text through stdin, run for at most 30 seconds, and never interpolate message text. These commands still affect the local Mac, so show a critical local warning and treat script configuration as privileged maintenance. Face recognition never satisfies this command policy.

CHAPTER 26

Treat model tools as proposals

The optional `robot_action` path validates a versioned proposal, routes it to the operator approval surface, correlates result messages, and returns only terminal outcomes to Gemini. Approval currently records operator intent; it does not prove physical execution exists.

Stage-originated Gemini turns reject non-stop physical tool calls even after the cue has timed out. `stop_motion` uses a priority path: stop coordinators, write one neutral/braked Base frame, drop the heartbeat, and return control authority to Brain. AMBER arm hold remains explicitly unverified.

A safe model architecture is:

```
model text/tool suggestion
        ↓ strict decoding
typed proposal
        ↓ policy and operator approval
bounded action state machine
        ↓ hardware-specific executor
measured result or explicit unavailable
```

Never connect model tokens directly to `write`, `ticcmd`, a servo target, shell command, or AMBER packet.

CHAPTER 27

Keep destination autonomy and learned traversability behind operator authority

Destination requests originate in the controller, resolve through bounded geocoding, and enter a local Valhalla-backed planning path. The current pilot limits travel to 50 m and rejects stale pose, map, lidar, depth, or authority state. Separately, the Myriad X sidewalk segmentation graph produces a bounded centerline/confidence observation at about 5 Hz. That observation can support a deterministic proportional steering policy only inside an explicitly authorized autonomy session.

`ROBTraversabilityRuntime` learns belly-camera ground context only from manually traversed and odometry-confirmed samples. Autonomous commands remain useful log evidence but never certify their own route. Manual takeover, explicit stop, session replacement/expiry, stale required sensing, or loss of controller authority ends or stops the motion path. A recognized person, a Messages sender, a VLM label, or a model's confident destination prose cannot open this gate.

CHAPTER 28

Stream camera video to Vision Pro separately

SOURCE TRAIL — ANALYZING NOW: `ROBVideoServer.swift`, `ROBVideoProtocol.swift`, `ROBCameraH264Encoder.swift`, `docs/vision-pro-video.md`, and Volume 7.

Cerebro advertises `_robvideo._udp` with ALPN `robvideo/1`, separately from `_robctl._udp` control. Both use TLS 1.3 and the paired certificate/secret relationship, but video uses its own authentication transcript and requires a matching live operator control session.

The service exposes three independent feed identifiers and pipelines: `front`, `belly`, and `insta360`. Front and belly negotiate at most 960 by 540; the panoramic feed uses 960 by 480. Each is capped at 20 fps and 1.5 Mbps, uses H.264 AVCC with B-frames disabled, and requests key frames at least every second. One physical provider connection and one bounded encoder/send state exist per active feed; subscribing to one does not authorize or start the others.

For each pipeline, one raw frame may wait, one encoder output may wait, and one send may be in flight. Congestion drops work instead of accumulating latency. Receiver recovery can request a key frame and codec configuration. A client opens the QUIC stream after authentication, and the server binds every subscription to the exact authenticated live control session. Loss of that session closes its media without taking the control listener down.

Volume 7 explains the other half: Vision Pro discovery, pinned identity, independent subscriptions, defensive framing, H.264 sample-buffer creation, flat front/belly/panorama windows, the inward-facing immersive sphere, controller ownership, dual-arm input, head pose, speech-to-text, and dead-man authority. Read the two books side by side whenever changing the wire contract.

CHAPTER 29

H.264 without hand-waving: pictures, access units, and NAL units

H.264 is a video coding format; it is not by itself a network protocol. The encoder turns images into coded syntax. The Network Abstraction Layer divides that syntax into **NAL units** suitable for storage or transport. One displayed picture is commonly represented by an **access unit**, which can contain several NAL units.

For H.264/AVC, the first byte of each NAL unit is:

```
bit 7        bits 6..5          bits 4..0
forbidden_0  nal_ref_idc        nal_unit_type
```

The current validator requires the forbidden bit to be zero and masks the low five bits to find the type. Useful types in ROB's profile include:

- **Type 1 – non-IDR coded slice:** predictive picture data that may depend on earlier reference pictures.
- **Type 5 – IDR coded slice:** a random-access recovery picture; the current key-frame flag must agree with the presence of type 5.
- **Type 6 – SEI:** supplemental information; not treated as the required video-coding payload.
- **Type 7 – SPS:** the sequence parameter set, carrying coded geometry, profile/level, and sequence-wide decoding facts.
- **Type 8 – PPS:** the picture parameter set, carrying picture-level decoding configuration referenced by slices.
- **Type 9 – access-unit delimiter:** an optional delimiter; ROB does not depend on it for framing.

SPS and PPS are configuration, not ordinary image pixels. Cerebro extracts them from `CMFormatDescription` on a key frame and sends them in a separate codec-configuration message. The receiver uses at least one SPS and one PPS to create `CMVideoFormatDescription`, then validates coded and presentation dimensions against the negotiated stream before allocating ongoing decode state.

Annex B and AVCC are two wrappers around NAL units

An Annex B byte stream finds NAL units with `00 00 01` or `00 00 00 01` start codes. An AVCC sample prefixes each NAL unit with a 1-, 2-, or 4-byte big-endian length. VideoToolbox gives ROB length-prefixed AVCC samples, and the exact prefix width comes from the format description.

```
AVCC access unit with 4-byte lengths

00 00 02 8A  [650-byte NAL unit]
00 00 00 31  [49-byte NAL unit]
```

The prefix is not part of the NAL unit. The first byte after each prefix is the NAL header. `validateLengthPrefixedAccessUnit` walks exactly this structure: read the configured prefix width, build a big-endian length, reject zero/truncation/overrun, inspect the NAL type, advance by exactly that length, and require at least one type 1–5 unit. A key-frame sample must contain type 5; a non-key sample must not claim it.

Do not scan an AVCC payload for Annex B start-code patterns. Compressed payload bytes can coincidentally contain them, and the representation already has authoritative lengths. Convert formats only at one named, tested boundary.

Read the encoder from input pixel to encoded bytes

`ROBCameraH264Encoder` performs this sequence:

1. Validate width, height, total pixels, frame rate, and bitrate against hard caps.
2. Create a VideoToolbox H.264 compression session whose destination pixels are NV12 video-range (`420YpCbCr8BiPlanarVideoRange`).
3. Request real-time operation, disable frame reordering, prefer Constrained Baseline, set expected frame rate/average bitrate, and request a key frame at least once per negotiated second.
4. Create one reusable pixel-transfer session in letterbox mode.
5. Admit samples at the requested rate using monotonic uptime. Do not encode every callback simply because the camera produced it.
6. Allocate from the compression session's pixel-buffer pool with a threshold of six. When the pool is exhausted, drop the frame and request a future recovery key frame.
7. Scale/letterbox the source into the destination buffer and call `VTCompressionSessionEncodeFrame` with a monotonic presentation time.
8. In the callback, claim completion exactly once because VideoToolbox can report a synchronous drop through more than one path.
9. Copy the bounded `CMBlockBuffer` bytes into owned `Data`; callback-owned memory must not escape by reference.
10. Read the not-sync attachment to determine whether VideoToolbox marked the sample as a key frame.
11. Extract the NAL length width from `CMFormatDescription`, validate every AVCC NAL, and on a key frame copy the parameter sets.
12. Emit sequence, wall-clock capture time, presentation time, duration, key-frame flag, optional parameter sets, prefix width, and payload.

Disabling B-frame reordering means decode order follows presentation order for this low-latency profile. It costs some compression efficiency but removes a reorder queue and simplifies recovery. Constrained Baseline is requested with documented fallback for unsupported properties, so runtime diagnostics should record the format actually returned.

ROB's transport is custom reliable QUIC, not RTP

RFC 6184 describes carrying H.264 NAL units over RTP using single-NAL packets, aggregation packets such as STAP-A, and fragmentation units such as FU-A. ROB's current path does none of those. It places complete AVCC access units inside the inner `RBVD` binary message, then places that message inside an outer ordered `RVID` frame on a reliable QUIC stream.

```
QUIC/TLS byte stream
  RVID 32-byte connection frame
    RBVD 92-byte media header
      AVCC access unit
        [length][NAL][length][NAL]...
```

The outer `RVID` layer identifies message kind and bounds the next payload before allocation. The inner `RBVD` layer binds media to a session UUID and subscription UUID and carries codec, sequence, capture time, presentation time,

duration, timescale, configuration generation, NAL-length width, key-frame flag, and payload length. The current caps are 64 KiB for configuration and 2 MiB for one access unit.

Reliable ordering preserves every byte, but a lost network packet can delay later video through head-of-line blocking. ROB limits the damage with a separate video connection, newest-only admission before encode, one queued output, one send in flight, deadlines, and stream teardown when a peer stops reading. Control uses a different QUIC connection, so a delayed video access unit does not sit in front of a stop message, although both still compete for Wi-Fi airtime.

Why a missing access unit triggers IDR recovery

Non-IDR slices may reference decoded pictures that the receiver no longer has. After a sequence gap, decoder error, renderer flush, or configuration-generation change, displaying more dependent frames can create corruption. The receiver enters `needsKeyFrame`, drops predictive units, rate-limits feedback, and waits for new configuration plus a valid type-5 IDR access unit. The server's encoder sets a thread-safe force-next-key-frame flag. Recovery is a state machine, not merely "send another picture."

Test NAL handling with generated fixtures: zero length, truncated prefix, oversized NAL, forbidden bit set, no VCL type, key flag without type 5, type 5 without key flag, SPS without PPS, dimension-changing SPS, configuration change without flush, sequence gap, and repeated key-frame requests. Run them without a camera or network before testing live media.

CHAPTER 30

Excavate the Kinect and OpenNI past

> **SOURCE TRAIL — ANALYZING NOW:** archived v2 commits `3dc82fd` through `23d8fbd` from 2018-01-12, archived v3's 2019 perception branches, and the current `ROBNiTEManager` and `TaskControllers` directories: `ROBNiTEManager.mm`, `FreenectPCL.mm`, `k2g.h`, `viewer.*`, shaders, and duplicated `* 2.*` artifacts.

The repository contains an older depth-vision stack built around Kinect/OpenNI/NiTE-style naming, libfreenect/PCL bridges, C++ point clouds, GL viewers, skeletal tracking, shaders, and serialized task-controller processes. The old archive now dates the first surviving OpenNI/PCL/NiTE commit sequence to 12 January 2018, with further libfreenect work in May 2018 and continuing VTK, NiTE, RealSense, human-tracking, and T265 experiments in 2019. The files arrived in the current Git history with the 2025 v5 import, but they were not created by that import.

The artifacts teach important lessons:

- camera SDKs age faster than domain concepts;
- C++ exception and ABI boundaries can destabilize an app process;
- duplicated filenames with `2` indicate migration debt, not alternate APIs to call casually;
- shader and viewer code may embody useful coordinate conventions even when its device path is retired;
- binary Core ML models such as `ObjectDetector.mlmodel` need provenance, input/output documentation, license review, and a reproducible evaluation set before reuse.

Do not delete ancient code merely because it looks strange. First map whether project files still compile it, whether runtime selectors reference it, whether a modern test covers its surviving behavior, and whether historical knowledge should move into documentation. Then retire it in a small commit with a rollback point.

CHAPTER 31

Compare Kinect-era and Luxonis-era design

The old path intertwined device SDK, C++ processing, visualization, and task launching. The new Luxonis path defines a provider contract and isolates the SDK in a supervised helper. The improvement is not simply "new camera has better depth." It is failure containment and replaceability.

Question	Historical depth stack	Current OAK design
Device boundary	native C++/framework integration	supervised Python process
Transfer	internal objects/task artifacts	versioned local `CDP1` bytes
RGB-depth pairing	implementation-specific	timestamp-sync and RGB alignment
Crash scope	potentially Cerebro process	helper can restart independently
Fallback	separate historical controllers	explicit AVFoundation RGB-only path
Contract tests	sparse artifacts	malformed/oversized IPC fixtures

The next evolution could replace Python with an XPC/native helper while preserving `CameraFrameSet`. Stable contracts let implementation technology change without rewriting perception and UI.

CHAPTER 32

Build a reliable light-saber battle trainer

SOURCE TRAIL – ANALYZING NOW: `ROBSaberChoreography.swift`, `StageShows/ProgressiveSaberTraining.robshow.json`, `GalacticSaberBattle.robshow.json`, `ROBAmberB1Kinematics.swift`, and `Tests/ROBSaberChoreographyFixtureTests.swift`.

The trainer uses named gestures and a progressive profile rather than embedding arbitrary joints in show files. `ROBSaberChoreography` maps an allow-listed gesture name to bounded Cartesian transforms and reports a requested duration. The real AMBER kinematics and calibration layers must still validate reachability and transform frames before execution.

A good training progression begins with guard pose and slow single-plane movements, then alternates direction, timing, and combinations. It should enforce a participant boundary, soft prop policy, low speed, reduced force, guarded robot workspace, independent emergency stop, and a dry-run visualization. "Battle" is theatrical choreography, never permission for autonomous contact.

The perception loop can score timing and approximate pose without commanding the arm from pixels. It observes participant pose, estimates phase, compares against the authored target, and chooses spoken coaching. Motion remains authored and bounded.

CHAPTER 33

Engineer the comedy show as a state machine

> **SOURCE TRAIL — ANALYZING NOW:** `ROBStageShowProtocol.swift`, the coordinator and window controller, the `OrbitusTenMinuteComedy` file under `StageShows`, and `ROBStageShowFixtureTests.swift` under `Tests`.

A `.robshow.json` document contains typed cues such as speech, wait, checkpoint, gesture, and Gemini turn. The codec rejects unknown or dangerous content. Show files cannot contain raw joint values, hosts, ports, or shell commands.

The coordinator advances one cue at a time and records what it is awaiting: speech completion, checkpoint, gesture, local model, Gemini, or timer. Each asynchronous request has an ID and deadline. Cancellation clears scheduled work and rejects late completion.

Four execution modes preserve rehearsability:

Mode	Local model	Gemini	Spoken fallback
Dry Run	no	no	none; validate without side effects
Offline	no	no	authored line
Local	optional MLX/llama.cpp	no	validated local line, then authored line
Adaptive	optional	yes	Gemini, validated local line, authored line

The comedy file targets roughly ten minutes and includes multiple adaptive moments. A clever line is never allowed to block the show forever: timeouts, authored fallback, and explicit cue advancement are part of the creative system.

CHAPTER 34

Test at contracts, not only windows

The repository includes fixture executables for Gemini messages, robot-action proposals, secure control, video framing and encoding, DepthAI IPC, local improvisation, stage shows, saber choreography, MLX stage observations, and catalog validity. These tests are valuable because most can run without moving ROB.

For every boundary, test:

1. smallest valid message;
2. largest valid message;
3. truncated header and payload;
4. oversized length before allocation;
5. unknown version, enum, or field;
6. stale sequence/generation/request ID;
7. cancellation before and after callback scheduling;
8. unavailable hardware/model/network;
9. repeated reconnect and shutdown;
10. redaction of secrets and private media.

Then perform a staged hardware test: no-power simulation, powered actuators mechanically isolated, wheels/treads raised, low limits, one subsystem, verified emergency stop, and only then integrated operation.

CHAPTER 35

Profile memory, latency, and thermals

Measure end-to-end age, not just inference duration. Attach capture timestamp to every frame and preserve it through RGB-D pairing, Vision, snapshot, model sampling, encoding, network transfer, and presentation.

Use Instruments Time Profiler, Allocations, Leaks, Metal System Trace, and Points of Interest. Add signposts around camera receive, RGB conversion, depth render, Vision request, VLM admission, model generation, H.264 encode, and main-thread scene update.

Watch for these characteristic failures:

- steadily rising SceneKit nodes or textures;
- a new `CIContext`, formatter, or pixel-buffer pool per frame;
- retained `CMSampleBuffer` objects in asynchronous closures;
- autoreleased Objective-C objects accumulating on worker threads;
- model containers loaded more than once;
- an unbounded array of diagnostics, semantic memories, or network frames;
- main-thread image conversion causing controller lag;
- repeated identical errors overwhelming Xcode and hiding state transitions.

Define production budgets for memory, frame age, UI frame time, model latency, camera restart time, and stop latency. A green light without those measurements is only a connection indication.

CHAPTER 36

Refactor without erasing history

The recovered archives make “without erasing history” literal. Preserve the v1 file fingerprints and the v2-v4 Git stores before removing copied SDKs, normalizing filenames, or consolidating targets. A clean modern repository and a faithful historical record are compatible only when migration boundaries are documented instead of mistaken for creation dates.

The highest-value seam is to split `ROBSerialBox` by device and responsibility:

```
ROBSerialBox façade for existing outlets/selectors├──
 ROBBaseSerialChannel├──
 ROBMaestroChannel├──
 ROBTicWaistController├──
 ROBAmberArmGatewayClient└──
 ROBControllerCommandArbiter
```

Move parsing and mapping into pure functions with fixtures first. Preserve the Objective-C façade so Interface Builder and existing callers keep working. Introduce typed state snapshots. Move every descriptor to one owning queue. Rate-limit diagnostics. Remove Head/Torso UI only after confirming no nib binding or operator workflow still requires it.

For camera work, preserve `CameraManagerProtocol` and `CameraFrameSet`. For model work, preserve `ROBLocalImprovisationProviding`, `SceneSnapshot`, and strict codecs. For Vision Pro work, preserve the versioned control/video contracts and update both repositories in lockstep.

CHAPTER 37

File-by-file reading route

Read in this order and make notes beside the code:

1. `README.md` and `docs/` for declared behavior and operational boundaries.
2. `AppDelegate.m` and `ROBMainViewController.mm` for process composition.
3. `ROBSerialBox.h/.m` for hardware history, current Base discovery, Maestro/Tic, and arm seams.
4. `CameraManager.swift`, `Webcam_color.py`, and the `ROBInsta360` services for three-role provider ownership and RGB-D/panoramic framing.
5. `CameraViewController.swift`, pose detector, and skeleton renderer for perception/display cost.
6. `ROBSceneSnapshot.swift` for typed world state and confidence.
7. `ROBRecordingCoordinator.swift` and its operational note before using any captured data.
8. `ROBFaceEmbeddingModel.swift`, gallery, recognition service, installer, and consent/threshold note together.
9. `ROBMLXRuntime.swift`, MLX provider, and local-improvisation protocol for private inference.
10. `ROBMessagesBridge.swift`, responder, transcript store, current-information services, and administrator-command policy as one isolated boundary.
11. `GeminiRoboticsProtocol.swift` before `ROBAI.swift`; learn the wire types before the session actor.
12. `ROBStageShowProtocol.swift` before its coordinator; learn allowed data before lifecycle.
13. `ROBSaberChoreography.swift`, AMBER kinematics, and visual calibration together.
14. `ROBVideoProtocol.swift`, encoder, and server, followed by Volume 7's three-feed receiver chapters.
15. Every matching fixture test before making a change.

CHAPTER 38

Production review checklist

- ☐ Every serial descriptor has one owner and bounded reconnect behavior.
- ☐ Base identity is detected by its existing firmware response without probe commands.
- ☐ Stale controller input produces one neutral frame and then drops heartbeat.
- ☐ Commanded, measured, and visually estimated poses remain distinct.
- ☐ Camera ownership cannot race between DepthAI and UVC providers.
- ☐ Face, belly, and Insta360 roles retain distinct device identity, sockets, demand, diagnostics, and calibration.
- ☐ Every IPC and network length is validated before allocation.
- ☐ Slow perception, inference, encoding, and network consumers retain newest-only bounded work.
- ☐ Vision and model work never blocks controller or main-thread responsiveness.
- ☐ SceneKit nodes, contexts, pixel buffers, and model containers remain bounded over long runs.
- ☐ Local and cloud model output crosses strict typed codecs.
- ☐ No language model has direct motor, shell, serial, or arm authority.
- ☐ Recording is explicit/recoverable, shows capacity and provenance, and never accepts autonomous self-labeling.
- ☐ Face enrollment/deletion is consented; profiles remain model-tagged/encrypted; thresholds, replays, and false accepts/rejects are tested; recognition grants no authority.
- ☐ Messages is disabled by default, exact-allowlisted, one-to-one and at-most-once; attachments, tools, memory, cloud disclosure, exports, and administrator scripts pass their fail-closed fixtures.
- ☐ Stage cues have deadlines, cancellation correlation, and authored fallback.
- ☐ Camera and microphone streaming show explicit effective operator state.
- ☐ Vision Pro control, arm, and three-feed video protocol versions match Volume 7's client.
- ☐ Historical Kinect artifacts are documented before removal or reuse.
- ☐ Tests pass before any powered-hardware commissioning.

CHAPTER 39

Closing principle

Cerebro is strongest when every subsystem admits uncertainty and owns failure locally. Serial discovery should identify rather than guess. Depth should mark invalid pixels. Vision should preserve confidence. Models should propose typed meaning rather than emit actuator bytes. Stage shows should continue through bounded fallback. Video should drop frames rather than delay control. The Vision Pro should express fresh human intent, while Cerebro and the robot enforce authority, time, and safe limits.

That is how a mixed Swift and Objective-C codebase becomes a dependable robot mind: not through one magical model, but through explicit contracts between imperfect parts.